

Contents

Beispiel-Bots — Erklärung für den Dozenten	1
RandomBot	1
StillstandBot	1
ChaserBot	2
FluchtBot	2
PowerBot	3
Gemeinsame Muster über alle Beispiel-Bots	5

Beispiel-Bots — Erklärung für den Dozenten

Detaillierte Erklärung der fünf fertigen Referenz-Bots unter `bots/examples/` (`src/main/kotlin/bots/examples/`). Diese Bots werden von Schülern nicht verändert — sie dienen als Testgegner (Sparring-Partner) und als Referenz, auf die man verweisen kann, ohne die Musterlösungen aus [loesungen.md](#) direkt zu zeigen.

Alle Bots implementieren `RobotBrain` und werden über `BotRegistry.kt` automatisch in die Bot-Auswahl der App eingebunden — es muss dafür nichts manuell registriert werden (anders als bei `teamXBots`).

RandomBot

```
class RandomBot(override val name: String = "RandomBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val direction = Direction.entries.random()
        return if (Random.nextBoolean()) {
            Action.Move(direction)
        } else {
            Action.Shoot(direction)
        }
    }
}
```

Was er macht: Wählt jeden Tick unabhängig voneinander eine zufällige Richtung (`Direction.entries.random()`) und eine zufällige Aktion (`Random.nextBoolean()` entscheidet Move vs. Shoot). Kein Bezug zu `sensors.self` oder `sensors.others` — der Bot “sieht” nichts, er würfelt nur.

Intention: Einfachste denkbare Baseline. Dient als unterster Maßstab: jeder von Schülern gebaute Bot sollte gegen `RandomBot` zuverlässig gewinnen, sonst stimmt etwas mit der eigenen Logik nicht. Auch nützlich als allererstes Testduell, weil er garantiert keine Exception wirft und sich immer “irgendwie” verhält.

Didaktischer Bezug: Entspricht fast 1:1 Story 1.1 (Musterlösung siehe [loesungen.md](#)), nur zusätzlich mit der Move/Shoot-Zufallsentscheidung kombiniert.

StillstandBot

```
class StillstandBot(
    override val name: String = "StillstandBot",
    private val shootDirection: Direction = Direction.EAST
) : RobotBrain {
```

```

    override fun decide(sensors: Sensors): Action {
        return Action.Shoot(shootDirection)
    }
}

```

Was er macht: Bewegt sich nie, schießt jeden Tick stur in dieselbe, im Konstruktor festgelegte Richtung (Default EAST). shootDirection ist ein optionaler Konstruktorparameter — im Turnier/Testduell wird i.d.R. der Default verwendet, aber man könnte in BotRegistry/Tests auch StillstandBot(shootDirection = Direction.NORTH) instanziiieren.

Intention: Kontrollierbarster mögliche Testgegner. Weil er sich nie bewegt, lässt sich exakt vorher-sagen, ob ein Schuss trifft — ideal, um zu prüfen, ob ein Schüler-Bot (a) überhaupt trifft, wenn er in Reihe/Spalte steht, und (b) einem Dauerfeuer ausweicht, wenn er selbst getroffen wird. Wird in Story 2.1 explizit als Testgegner genannt.

Didaktischer Bezug: Entspricht direkt Story 2.1.

ChaserBot

```

class ChaserBot(override val name: String = "ChaserBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val target = sensors.nearestEnemy() ?: return Action.Wait
        val self = sensors.self.position

        return if (sensors.canShoot(target)) {
            Action.Shoot(self.directionTo(target.position)!!)
        } else {
            Action.Move(self.approachDirectionTo(target.position)!!)
        }
    }
}

```

Was er macht: - sensors.nearestEnemy() (Toolkit-Funktion, siehe [toolkit-referenz.md](#)) sucht per Manhattan-Distanz den nächstgelegenen lebenden Gegner und liefert null, wenn keiner mehr lebt — decide() fängt das mit ?: return Action.Wait ab, statt eine Exception zu werfen. - sensors.canShoot(target) prüft, ob der Bot bereits in Reihe/Spalte mit dem Ziel steht (Sichtlinie). Ist das der Fall, liefert directionTo die exakte Schussrichtung. - Steht das Ziel nicht in Sichtlinie, übernimmt approachDirectionTo die Annäherung: die Achse mit dem größeren Ausschlag wird zuerst angeglichen. Das führt über mehrere Ticks zu einer treppenartigen (“diagonalen”) Annäherung, bis eine der beiden Differenzen 0 erreicht und ab dann geschossen wird.

Intention: Zeigt die “Angriff”-Grundlogik aus Epic 2 (Sichtlinie prüfen -> schießen, sonst verfolgen) in einer sauberen, vollständigen Form. Aggressiv, aber ohne jede Rücksicht auf eigene Gesundheit — stirbt ggf. lieber angreifend, als zu fliehen.

Didaktischer Bezug: Ist die “exakte, bereits getestete Referenz” für die Stories 2.2/2.3 (siehe Verweis in loesungen.md). Zeigt Schülern gleichzeitig, wie kompakt eine Lösung mit den Toolkit-Funktionen werden kann.

FluchtBot

```

class FluchtBot(override val name: String = "FluchtBot") : RobotBrain {

```

```

private companion object {
    const val FLEE_HEALTH_THRESHOLD = 20
}

override fun decide(sensors: Sensors): Action {
    val target = sensors.nearestEnemy() ?: return Action.Wait
    val self = sensors.self.position

    return if (sensors.self.health < FLEE_HEALTH_THRESHOLD) {
        Action.Move(self.fleeDirectionFrom(target.position)!!)
    } else if (sensors.canShoot(target)) {
        Action.Shoot(self.directionTo(target.position)!!)
    } else {
        Action.Move(self.approachDirectionTo(target.position)!!)
    }
}
}

```

Was er macht: - Solange die eigenen HP bei 20 oder mehr liegen, verhält sich FluchtBot wie ChaserBot (canShoot prüft Sichtlinie, approachDirectionTo nähert sich sonst an). - Fällt die Gesundheit unter FLEE_HEALTH_THRESHOLD, übernimmt fleeDirectionFrom die Fluchtrichtung — die Umkehrung von approachDirectionTo, reine Achsen-Heuristik (Achse mit größerem Ausschlag zuerst, dann in die Gegenrichtung bewegen). - Landet der Bot dabei an einer Wand, ist das kein Problem: die Engine blockt ungültige Bewegungen ohnehin ab (resolveMoves in GameEngine.kt), FluchtBot selbst muss das nicht extra prüfen.

Intention: Zeigt die einfachste Kombination aus Angriff (Epic 2) und Flucht bei niedriger Gesundheit (Story 1.3). fleeDirectionFrom ist bewusst die einfache Einstiegsvariante — kein Durchprobieren aller vier Richtungen, kein Eckfall-Check. Genau das ist der Maßstab, den Schüler mit der Toolkit-Funktion erreichen sollen.

Didaktischer Bezug: Referenz für Story 1.3 kombiniert mit Epic 2. Wer eine robustere Fluchtstrategie bauen will (z.B. alle 4 Richtungen simulieren, Eckfall behandeln), kann das in Story 3.3 (Kür) selbst ergänzen — dafür bräuchte man zusätzlich Position.moved() und isInsideArena() aus dem Toolkit.

PowerBot

```

class PowerBot(override val name: String = "PowerBot") : RobotBrain {

    override fun decide(sensors: Sensors): Action {
        val self = sensors.self.position
        if (sensors.others.isEmpty()) return Action.Wait

        val weakestAligned = sensors.enemiesInLineOfSight().weakest()
        if (weakestAligned != null) {
            return Action.Shoot(self.directionTo(weakestAligned.position)!!)
        }

        val target = bestTarget(sensors)
        return moveTowardAlignment(self, target.position)
    }

    private fun bestTarget(sensors: Sensors): RobotState =

```

```

sensors.others.minBy { it.position.manhattanDistanceTo(sensors.self.position) }

private fun moveTowardAlignment(self: Position, target: Position): Action {
    val dx = target.x - self.x
    return if (dx != 0) {
        Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
    } else {
        Action.Move(self.approachDirectionTo(target)!!)
    }
}
}
}

```

Was er macht – Schritt für Schritt: 1. **Sichtlinie zuerst prüfen, aber mit Zielpriorisierung:** `sensors.enemiesInLineOfSight()` (Toolkit-Funktion) filtert alle Gegner, die bereits in Reihe/Spalte stehen. Stehen mehrere davon in Sichtlinie, wird nicht einfach der nächste genommen (wie bei `ChaserBot`), sondern per `weakest()` (ebenfalls Toolkit) der mit den wenigsten HP gewählt – schwächsten zuerst töten, praktisch die direkte Umsetzung der Musterlösung zu Story 3.2 (Zielpriorisierung), nur in Kombination mit Sichtlinien-Filterung. 2. **Kein Gegner in Sichtlinie -> Ziel per Score wählen:** `bestTarget` berechnet für jeden Gegner einen kombinierten Score aus Distanz (`manhattanDistanceTo`, Toolkit) und Gesundheit (`it.health / 10`) und wählt das Minimum – nahe UND schwache Gegner werden bevorzugt, nicht nur die kleinste Distanz wie bei `ChaserBot`. Die Gewichtung `health / 10` ist eine bewusst gewählte Konstante (bei `startHealth = 100` reicht die Health-Spanne 0..100, geteilt durch 10 also 0..10 – vergleichbar mit typischen Distanzwerten auf einem 10x10-Feld), keine “offizielle” Formel, sondern ein empirisch funktionierender Kompromiss. 3. **Ausrichten:** `moveTowardAlignment` schließt zuerst die X-Differenz (peilt dieselbe Spalte an) von Hand, erst wenn `dx == 0`, übernimmt `approachDirectionTo` den Rest (Y-Achse). Anders als bei `ChaserBot` (größere Achse zuerst, komplett über `approachDirectionTo`) ist hier die Reihenfolge fest “X vor Y” – laut Doc-Kommentar im Quellcode empirisch als stärkste Annäherungsstrategie gegen die anderen Beispiel-Bots getestet. Das ist auch der Grund, warum `PowerBot` hier bewusst nicht komplett auf `approachDirectionTo` setzt. 4. **Bewusst keine Flucht.** Anders als `FluchtBot` hat `PowerBot` keinen Health-Schwellenwert, unter dem er flieht. Das ist eine explizite Design-Entscheidung, keine vergessene Story: gegen einen Gegner mit demselben Fluchtmuster (z.B. `FluchtBot` selbst) würde ein Flucht-Trigger dazu führen, dass sich beide Bots synchron im Kreis jagen, ohne sich je zu treffen -> Unentschieden statt Sieg. Aggressiv bleiben ist hier die bessere Strategie.

Bekannte Grenze (kein Bug): Gegen einen exakt gleich starken, ähnlich aggressiven Bot (gleiche Start-HP, gleicher Schaden pro Treffer, z.B. `ChaserBot`) endet der Kampf zwangsläufig unentschieden – stehen beide in derselben Linie und feuern gleichzeitig, sterben sie bei `startHealth = 100` und `damagePerHit = 10` nach exakt 10 gegenseitigen Treffern gleichzeitig. Das ist schlicht Arithmetik der Engine-Defaults, keine Schwäche der Zielwahl-Logik.

Intention: Der aggressivste und “klügste” Beispiel-Bot – kombiniert Zielpriorisierung (Epic 3) mit einer verfeinerten Bewegungsstrategie. Dient als starker Endgegner/Benchmark: Schüler-Teams, deren Bot regelmäßig gegen `PowerBot` gewinnt, haben vermutlich eine sehr solide Strategie gebaut.

Didaktischer Bezug: Gute Referenz für Story 3.2 (Zielpriorisierung) in einer fortgeschritteneren Form (Score aus zwei Kriterien statt nur einem). Auch geeignet, um im Review mit stärkeren Teams über Grenzen von Heuristiken zu sprechen (z.B. die “Gleichstand endet unentschieden”-Eigenschaft als Diskussionsanlass für Story 3.3, die Kür-Aufgabe).

Gemeinsame Muster über alle Beispiel-Bots

- **Toolkit-Funktionen aus `framework.arena`** (`nearestEnemy`, `canShoot`, `directionTo`, `approachDirectionTo`, `fleeDirectionFrom`, `weakest`, `enemiesInLineOfSight`, `manhattanDistanceTo`, ...) statt Von-Hand-Berechnung — taucht in jedem Bot außer `RandomBot`/`StillstandBot` auf. Volle Referenz: [toolkit-referenz.md](#).
- **Manhattan-Distanz** (nicht euklidisch) ist die Grundlage aller Distanz-Funktionen im Toolkit — passend zum Grid, auf dem nur orthogonale Bewegungen möglich sind.
- **null-Behandlung statt !!** bei der Gegnersuche (`nearestEnemy()`, `weakest()`, `enemiesInLineOfSight().weakest()` liefern null/leere Liste statt zu crashen). Ein !! ist nur dort vertretbar, wo vorher explizit auf Sichtlinie/Nicht-Leerheit geprüft wurde (z.B. `directionTo(...)`!!, nachdem `canShoot(...)` schon `true` war).
- **Y-Achse wächst nach unten** (Bildschirmkoordinaten, siehe Kommentar in `Models.kt:6`): `NORTH` bedeutet `y - 1`, nicht `y + 1`. Häufigste Verwechslungsquelle bei Schülern — beim Review gegenprüfen, auch wenn die Toolkit-Funktionen das Vorzeichen-Denken jetzt übernehmen.
- **Keine Bots werfen Exceptions bei leerem `sensors.others`** (alle haben einen frühen `return Action.Wait`-Pfad) — das ist auch ein explizites Akzeptanzkriterium in mehreren Backlog-Storys (z.B. 1.3) und sollte bei Schüler-Code genauso eingefordert werden.